



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

Lucrare de licență

DEZVOLTAREA UNUI NUCLEU DE SISTEM DE OPERARE

Absolvent

Dumitru Georgian-Fabian

Coordonator științific

Lect.dr. Drăgulici Dumitru Daniel

București, septembrie 2025

Rezumat

Lucrarea prezintă dezvoltarea unui nucleu minimalist de sistem de operare pentru arhitectura x86-64, implementat în C++ și assembly. Nucleul are structură monolitică și suportă multi-tasking preemptiv pe un singur nucleu, cu procese izolate în spații de memorie virtuală. Sunt detaliate subsisteme precum gestionarea memoriei, planificatorul Round-Robin, întreruperile, terminalul virtual și sistemul de fișiere virtual (VFS). În spațiul utilizator sunt incluse programe de bază, un shell simplu și compilatorul TCC, care permite rularea aplicațiilor direct în sistem, demonstrând caracterul self-hosting al nucleului. Proiectul are scop educațional, urmărind claritatea implementării, iar direcțiile viitoare vizează suport multi-core, drivere moderne și extinderea sistemelor de fișiere.

Abstract

This thesis describes the development of a minimalist operating system kernel for the x86-64 architecture, implemented in C++ with minimal assembly. The kernel is monolithic, supports preemptive multitasking on a single core, and isolates processes in virtual memory spaces. Core subsystems include memory management, the Round-Robin scheduler, interrupt handling, the virtual terminal, and a virtual file system (VFS). User space features a basic shell and the TCC compiler, enabling application development and execution directly within the system, thus proving its self-hosting capability. The project has an educational purpose, focusing on clarity of implementation, with future work aimed at multi-core support, modern device drivers, and extended file system functionality.

Cuprins

1	Introducere	4
1.1	Motivație	4
1.2	Scopul lucrării	5
1.3	Structura lucrării	5
2	Preliminarii	6
2.1	Arhitectura x86-64 și modelul de execuție	6
2.2	Firmware-ul UEFI și rolul său în inițializarea sistemului	7
2.3	Limine – bootloader-ul sistemului	8
2.4	Rolul nucleului într-un sistem de operare	9
2.5	Spațiul utilizator	10
3	Design-ul nucleului	11
3.1	Subsisteme ale nucleului	12
3.1.1	Managerul memoriei fizice	12
3.1.2	Managerul memoriei virtuale	13
3.1.3	Heap-ul nucleului	14
3.1.4	Gestionarea întreruperilor	15
3.1.5	Mecanisme de temporizare	16
3.1.6	Planificatorul și comutare de context	16
3.1.7	Subsistemul de input	18
3.1.8	Terminalul virtual	19
3.1.9	Sistemul virtual de fișiere	20
3.2	Inițializarea nucleului și lansarea primului proces	21
4	Spațiul utilizator	24
4.1	Apelurile de sistem	24
4.2	Crearea și execuția programelor	25
4.3	Ecosistemul minimal de programe	26
5	Concluzii	27
5.1	Rezultate obținute	27
5.2	Limitările nucleului	27
5.3	Direcții viitoare	28
	Bibliografie	29

Capitolul 1

Introducere

1.1 Motivație

Un sistem de operare este un program de bază care gestionează resursele hardware și software ale unui calculator și oferă servicii comune pentru rularea aplicațiilor, facilitând astfel interacțiunea între programe și componentele fizice ale sistemului. La baza unui astfel de sistem se află **nucleul** (*kernel*), componenta centrală care implementează efectiv mecanismele fundamentale, precum gestionarea memoriei, planificarea și comutarea proceselor sau accesul la sistemele de fișiere. Nucleul asigură funcționarea corectă a întregului sistem și oferă un mediu stabil pentru rularea aplicațiilor.

Motivația principală a acestei lucrări a fost **curiozitatea intelectuală** legată de funcționarea unui sistem de operare în detaliu. Consider că implementarea practică **de la zero** oferă o înțelegere mult mai profundă decât simplul studiu al conceptelor teoretice ale sistemelor de operare. Prin dezvoltarea propriului sistem, te confrunți în mod direct cu probleme deja întâlnite și rezolvate anterior de dezvoltatori, iar abordarea lor practică permite o perspectivă mult mai clară a motivelor pentru care mecanismele și conceptele au evoluat așa cum sunt astăzi, fiind modelate de constrângeri atât hardware, cât și software.

O altă motivație importantă a fost dorința de a avea **control direct asupra calculatorului meu personal**, asupra procesorului, memoriei și a altor componente. În sistemele de operare moderne, interacțiunea directă cu hardware-ul este aproape imposibilă din cauza multiplelor straturi de abstracție, iar calculatorul se simte mai degrabă ca fiind deținut de sistemul de operare decât de utilizator. Prin dezvoltarea unui nucleu propriu, am putut experimenta și manipula resursele hardware fără intermediari, obținând o experiență apropiată de cea oferită de sistemele *embedded*, unde controlul asupra calculatorului este complet al utilizatorului.

1.2 Scopul lucrării

Scopul acestei lucrări constă în dezvoltarea unui **nucleu** pentru arhitectura **x86-64**, utilizând limbajul C++ împreună cu porțiuni minimale de *assembly*, însoțit de un set restrâns de programe exemplificative care să permită testarea și demonstrarea funcționalității nucleului. Proiectul urmărește crearea unui mediu **educațional și practic**, care să ofere o înțelegere aprofundată a mecanismelor fundamentale ale unui sistem de operare modern, construit **de la zero**.

Lucrarea implementează un sistem cu **multitasking preemptiv** pe un singur nucleu, în care procesele sunt izolate în propriul context de memorie, utilizând suportul hardware oferit de procesor. Procesele rulează în **spațiu utilizator**, iar nucleul gestionează planificarea și comutarea contextului, asigurând un mediu stabil și controlat. Totodată, nucleul include un **sistem virtual de fișiere (VFS)**, care abstractizează operațiile asupra fișierelor, independent de structura concretă a unui sistem de fișiere fizic, permițând extinderea și adaptarea ulterioară a sistemului.

Un obiectiv central al proiectului a fost ca nucleul să fie suficient de complet pentru a permite portarea unui **compiler de C** și rularea programelor dezvoltate direct în interiorul sistemului, în regim *self-hosting*. Această caracteristică evidențiază faptul că nucleul, deși minimalist, oferă mecanismele esențiale pentru execuția aplicațiilor și manipularea resurselor, demonstrând astfel **funcționalitatea practică și extensibilitatea** sistemului.

Lucrarea nu a urmărit optimizarea performanței sau implementarea tehnicilor avansate de securitate și nu și-a propus să creeze un sistem complet unic. Dimpotrivă, nucleul a fost conceput să urmeze cât mai îndeaproape structura și principiile nucleului **Linux** la nivelul interfeței între utilizator și nucleu, pentru a facilita portarea programelor și a oferi un mediu familiar, ușor de utilizat și de experimentat.

1.3 Structura lucrării

Lucrarea este organizată în cinci capitole principale. Primul capitol introduce motivația și scopul proiectului, oferind contextul general și justificarea dezvoltării nucleului, evidențiind atât abordarea educațională, cât și cea practică a proiectului. Al doilea capitol prezintă conceptele fundamentale necesare pentru înțelegerea lucrării, incluzând arhitectura x86-64, UEFI, bootloader-ul, precum și aspecte generale despre nucleu și spațiul utilizator.

Capitolul trei detaliază, în principal, design-ul nucleului, precum și diferitele subsisteme, prezentând rolul acestora, câteva aspecte legate de implementare și modul în care interacționează între ele. Tot aici este descris fluxul de la bootloader până la rularea primului program în spațiul utilizator. Capitolul patru ilustrează modul în care programele pot fi dezvoltate și executate pe sistem, evidențiind interfața dintre utilizator și nucleu. De asemenea, sunt discutate aspecte legate de portarea aplicațiilor.

Ultimul capitol cuprinde concluziile lucrării, prezentând realizările obținute, funcționalitățile care nu au fost implementate și oferind o reflecție asupra cunoștințelor acumulate, precum și direcțiile viitoare de dezvoltare.

Capitolul 2

Preliminarii

2.1 Arhitectura x86-64 și modelul de execuție

Arhitectura x86-64 (cunoscută și ca **AMD64** [1]) este platforma de procesoare pe care rulează **majoritatea calculatoarelor** moderne, în special desktop-uri. Aceasta definește regulile de funcționare ale procesorului, incluzând setul de instrucțiuni, modelul memoriei și mecanismele de protecție asociate.

Procesorul x86-64 operează în mai multe moduri, dintre care cel mai relevant pentru sistemele moderne este **64-bit long mode**, ce permite utilizarea completă a regiștrilor pe 64 de biți și nu păstrează compatibilitatea cu codul pe 32 de biți¹. În acest mod, procesorul pune la dispoziție 16 regiștri generali pe 64 de biți: **RAX**, **RCX**, **RDX**, **RBX**, **RSP**, **RBP**, **RSI**, **RDI**, **R8-R15**, precum și regiștri speciali precum **RIP** (registru care conține adresa instrucțiunii următoare) și **RFLAGS** (registru care conține flaguri ce reflectă starea procesorului după execuția instrucțiunilor). De asemenea, arhitectura definește regiștri de segment: **CS**, **SS**, **DS**, **ES**, **FS**, **GS** și regiștri de control **CR0-CR8** care sunt esențiali pentru gestionarea memoriei și a privilegiilor.

Modelul de memorie în x86-64 este unul *linear*, fără segmentare tradițională. Procesorul nu lucrează direct cu memoria fizică, ci utilizează un strat intermediar implementat la nivel hardware numit **Memory Management Unit (MMU)**, care translatează adresele virtuale accesate de procesor în adrese fizice din RAM și permite astfel implementarea memoriei virtuale și izolarea proceselor. Memoria este împărțită în pagini de dimensiuni tipice de 4 KiB, iar traducerea se realizează printr-o ierarhie de tabele de pagini (**PML4**, **PDPT**, **PD**, **PT**) configurată prin registrul de control **CR3**. Din spațiul virtual teoretic de 2^{64} adrese, doar primele 2^{48} sunt utilizate în practică².

Procesorul definește patru nivele de privilegiu, denumite *ring-uri*, dintre care cel mai important este **ring 0** (nivelul nucleului) care are privilegiu maxim, cu acces complet la instrucțiuni și hardware și **ring 3** (nivelul utilizator) unde rulează aplicațiile obișnuite, având acces limitat la setul de instrucțiuni.

¹Long mode include submoduri precum un mod care pastreaza compatibilitatea cu cod pe 32 de biti și modul de 64 de biți nativ.

²Există totuși extensii precum **PML5** care permit spații mai mari, dar acestea nu sunt comune.

Arhitectura x86-64 include și structuri critice pentru gestionarea întreruperilor și a trecerii între nivele de privilegii. **Interrupt Descriptor Table (IDT)** leagă întreruperile de *handler*-ele³ corespunzătoare și suportă 256 de intrări, adresa acestei structuri trebuie să fie încărcată în registrul IDTR. Similar, **Global Descriptor Table (GDT)** definește zone de cod și date, specificând privilegiile și alte atribute relevante, cum ar fi *Descriptor Privilege Level (DPL)*, utilizate pentru tranzițiile între ring-uri. Regiștrii de segment conțin indici referitori la această tabelă. În GDT se regăsește și structura **Task State Segment (TSS)**, care permite schimbarea în mod atomic, la nivel hardware, a stivei (indicată de registrul RSP) atunci când se produce o tranziție a nivelului de privilegiu, de exemplu la momentul întreruperilor. Acest mecanism este esențial pentru implementarea comutării contextului.

2.2 Firmware-ul UEFI și rolul său în inițializarea sistemului

Unified Extensible Firmware Interface (UEFI) [2] reprezintă standardul modern pentru firmware-ul sistemelor de calcul, înlocuind treptat vechiul BIOS. Rolul său esențial este de a inițializa componentele hardware în timpul pornirii și de a furniza un mediu standardizat prin care controlul poate fi transferat către sistemul de operare. Spre deosebire de BIOS, UEFI rulează nativ în mod pe 64 de biți, ceea ce permite utilizarea completă a resurselor procesorului și oferă suport extins pentru operații pe arhitecturi moderne.

În faza de inițializare, firmware-ul UEFI detectează și configurează dispozitivele hardware majore — procesorul, memoria RAM, controlerele de stocare și interfețele grafice — construind totodată o **hartă a memoriei**. Aceasta indică zonele disponibile pentru utilizare, precum și regiunile rezervate sau protejate. Informația este crucială pentru kernel, care trebuie să configureze corect mecanismul de mapare a adreselor virtuale în adrese fizice.

Standardul definește un set de *servicii*, împărțite în două categorii:

- **Boot services** – sunt disponibile pe durata procesului de inițializare și oferă funcționalități precum citirea din dispozitive de stocare, manipularea memoriei și gestionarea întreruperilor. După ce nucleul preia controlul, aceste servicii sunt dezactivate.
- **Runtime services** – rămân accesibile și după finalizarea secvenței de boot, permițând operații precum citirea ceasului în timp real, gestionarea variabilelor UEFI sau controlul anumitor periferice.

Printre elementele introduse de UEFI se numără și mecanismele moderne de securitate. **Secure Boot** verifică semnăturile digitale ale codului executabil pentru a preveni încărcarea software-ului neautorizat sau malițios, garantând că numai programe valide, semnate corespunzător, pot fi executate înainte de transferul controlului către sistemul de operare.

Prin aceste caracteristici, UEFI furnizează un mediu robust și sigur pentru inițializarea hardware-ului și pentru transmiterea informațiilor esențiale către *kernel*, facilitând intrarea procesorului în **long mode** pe 64 de biți cu acces complet la resurse și protecție adecvată a memoriei.

³handler – rutină executată de procesor, în general, apelată în urma unui eveniment

2.3 Limine – bootloader-ul sistemului

Bootloader-ul reprezintă componenta intermediară dintre firmware-ul UEFI și nucleu, având rolul de a pregăti mediul de execuție și de a încărca kernelul în memoria sistemului. În cazul acestui proiect, a fost ales **Limine** [3], un bootloader modern proiectat special pentru a funcționa cu UEFI și pentru a oferi o interfață standardizată și flexibilă între firmware și kernel.

Spre deosebire de un simplu mecanism de încărcare a unui executabil în RAM, Limine garantează un set de proprietăți esențiale mediului în care nucleul își începe execuția. Aceste garanții includ, printre altele, maparea kernelului în *higher-half memory*⁴, configurarea registrelor și a spațiului de adrese pentru rulare directă în *long mode* pe 64 de biți, precum și transmiterea unei hărți detaliate a memoriei fizice. Aceste caracteristici asigură faptul că nucleul este plasat într-un context de execuție predictibil, indiferent de particularitățile firmware-ului de pe fiecare sistem.

În plus, Limine expune către nucleu o serie de structuri standardizate care facilitează inițierea componentelor critice. Printre acestea se numără structura *framebuffer*⁵, ce oferă acces direct la memoria video pentru afișarea de informații vizuale în stadiile incipiente ale pornirii, structura **SMP**⁶, necesară pentru gestionarea și sincronizarea procesoarelor într-un sistem multiprocesor, precum și informații despre modulele încărcate sau alte resurse puse la dispoziție încă din faza de boot.

Prin aceste mecanisme, kernelul beneficiază nu doar de faptul că este încărcat corect în RAM, ci și de un cadru de execuție stabil, coerent și standardizat. Astfel, subsistemele interne pot fi inițializate într-o manieră robustă, fără a depinde de detalii variabile ale implementării firmware-ului, ci bazându-se pe garanțiile clare și uniforme pe care le oferă bootloader-ul.

⁴*Higher-half memory* desemnează maparea nucleului într-o regiune superioară a memoriei virtuale, separată de aplicațiile utilizator.

⁵*Framebuffer*-ul este o zonă de memorie folosită pentru scriere directă a pixelilor pe ecran.

⁶**SMP** (*Symmetric Multiprocessing*) permite rularea simultană a mai multor procesoare pe același sistem.

2.4 Rolul nucleului într-un sistem de operare

Nucleul (*kernel*) reprezintă componenta centrală a unui sistem de operare, responsabilă pentru gestionarea resurselor hardware și pentru furnizarea unui mediu controlat și sigur în care rulează aplicațiile. În esență, nucleul asigură interfața dintre programe și componentele fizice ale sistemului, coordonând activitatea procesorului, memoriei, dispozitivelor periferice și a altor resurse critice.

Funcțiile nucleului pot fi grupate în mai multe domenii principale:

- **Gestionarea proceselor:** Nucleul creează, planifică și comută procesele, asigurând multitasking și izolarea fiecăruia în propriul context. Printre funcțiile concrete se numără crearea de procese, terminarea lor, comutarea contextului și implementarea mecanismelor de sincronizare și comunicare între procese (*inter-process communication*).
- **Gestionarea memoriei:** Nucleul alocă și eliberează memorie pentru procese, gestionează memoria virtuală și protecția accesului, permițând fiecărui proces să primească un spațiu de adresare propriu, iar accesul neautorizat la zonele altor procese să fie prevenit.
- **Administrarea temporizatoarelor și timpului (*timers*):** Nucleul folosește temporizatoare pentru planificare, măsurarea timpului de execuție și gestionarea evenimentelor periodice, permițând crearea de mecanisme de temporizare precise.
- **Sistemul de fișiere:** Nucleul furnizează mecanisme pentru crearea, citirea, scrierea și manipularea fișierelor și directoarelor, abstractizând dispozitivele de stocare și oferind un model uniform de acces la date.
- **Gestionarea dispozitivelor periferice:** Nucleul interacționează cu hardware-ul prin drivere, oferind aplicațiilor un mod standardizat de a utiliza tastaturi, plăci video, dispozitive de stocare, plăci de rețea și altele.
- **Suport pentru comunicarea în rețea:** Nucleul include subsistemul de rețea, care gestionează transmiterea și recepția datelor între dispozitive locale și externe, oferind interfețe standardizate pentru protocoale precum TCP sau UDP.
- **Securitate și protecție:** Nucleul controlează accesul la resurse, implementează politici de securitate și asigură izolarea proceselor pentru a preveni erori sau atacuri.

Prin urmare, având în vedere diversitatea funcțiilor și a subsistemelor pe care le coordonează, nucleul reprezintă un program complex, iar realizarea unei implementări riguroase și funcționale necesită un efort considerabil și o atenție deosebită la detalii.

2.5 Spațiul utilizator

Spațiul utilizator (*user-space*) reprezintă mediul de execuție în care rulează programele obișnuite, fără privilegii speciale, distinct de nucleu, care operează în **ring 0**. Aplicațiile din *user-space* funcționează în **ring 3**, având acces controlat la resurse și instrucțiuni privilegiate, ceea ce garantează **stabilitatea și protecția sistemului**.

În acest spațiu rulează diverse programe: utilitare de sistem, shell-uri, aplicații grafice și servicii de fundal. Pentru a interacționa cu resursele critice, aceste programe folosesc **apeluri de sistem** (*system calls*), prin care solicită operații precum alocarea de memorie, citirea și scrierea fișierelor, planificarea proceselor, gestionarea temporizatoarelor sau comunicarea prin rețea. Fiecare cerere este verificată și executată într-un mod sigur, menținând integritatea resurselor și izolarea proceselor.

Pentru a facilita programarea, spațiul utilizator include biblioteci și rutine standard care înfășoară aceste apeluri. Funcții precum **open**, **read**, **write** sau **send/recv** permit accesul la funcționalități complexe fără a expune detalii hardware, creând un strat de abstractizare sigur și coerent. Astfel, programele pot rula fără a compromite securitatea sau stabilitatea sistemului.

Din perspectiva utilizatorului, interacțiunea cu sistemul de operare se realizează exclusiv prin aplicațiile din *user-space*. Acest spațiu servește ca interfață controlată între resurse și utilizator, oferind un mediu predictibil pentru rularea programelor, fără ca detaliile interne ale nucleului să fie vizibile sau accesibile direct.

Capitolul 3

Design-ul nucleului

În proiectarea unui nucleu, există în general două modele principale: nucleeele **monolitice**, cum este cazul Linux¹, și **micro-nucleeele**, care păstrează nucleul minimal și mută subsistemele în *user-space*, nucleul funcționând ca un *proxy* pentru comunicarea între acestea. Diferențele dintre aceste modele sunt vaste, însă pentru acest proiect a fost ales modelul **monolitic**, datorită simplității implementării și a controlului total oferit asupra resurselor sistemului.

Pentru execuție, nucleul utilizează un model de **preemptive multitasking** pe un singur nucleu de procesor. Această alegere oferă un echilibru între complexitatea datorată comutării de context și evitarea problemelor asociate cu *multicore*-ul, precum condițiile de cursă sau *deadlock*-urile. Modelul ales permite planificarea atât a proceselor din *user-space*, cât și a thread-urilor din nucleu, fără a complica excesiv arhitectura.

Memoria virtuală a fost utilizată astfel încât fiecare proces să ruleze într-un **spațiu de memorie izolat**, garantând protecția și stabilitatea sistemului. Planificatorul nucleului este implementat printr-un mecanism *round-robin* asupra listei de procese gata de execuție, oferind o planificare simplă și predictibilă. Această abordare permite o înțelegere clară a mecanismului de multitasking și facilitează extinderea ulterioară a funcționalităților.

O componentă esențială inclusă în nucleu este **sistemul de fișiere virtual (VFS)**, care oferă o interfață unificată pentru diferite implementări de sisteme de fișiere, cum ar fi **EXT2** [4] sau *ramfs*². VFS permite nucleului să gestioneze fișiere și directoare într-un mod abstract, independent de implementarea concretă a sistemului de fișiere fizic.

Pentru simplitate, nucleul nu implementează conceptele legate de utilizatori; practic, există un singur utilizator, echivalentul *root*³, iar toate procesele rulează cu aceleași privilegii. Această alegere simplifică gestionarea drepturilor și reduce complexitatea inițială a implementării.

¹Linux este un exemplu clasic de nucleu monolitic, în care majoritatea subsistemelor sunt integrate direct în nucleu.

²Sistem de fișiere simplu care stochează fișierele direct în RAM, utilizat pentru fișiere temporare.

³Utilizatorul cu privilegii maxime, care poate accesa și modifica toate resursele sistemului.

3.1 Subsisteme ale nucleului

Nucleul poate fi împărțit în mai multe **subsisteme** sau componente, care lucrează împreună pentru a asigura funcționarea sistemului de operare. O parte semnificativă a dificultății acestei lucrări a constat în dezvoltarea și evoluția paralelă a acestor componente. Datorită interdependențelor dintre subsisteme, implementarea unuia depinde adesea de existența celorlalte. De exemplu, crearea unui planificator solid necesită acces la un mecanism prin care pot fi încărcate executabile în memorie, ceea ce implică existența unui **sistem de fișiere** funcțional. La rândul său, pentru a testa și valida funcționalitatea sistemului de fișiere într-un mod dinamic, este necesară existența de procese care să exercite diferite funcționalități. Astfel, toate subsistemele au fost dezvoltate și îmbunătățite concomitent, cu ajustări și optimizări reciproce.

În continuare, vor fi prezentate subsistemele considerate esențiale, descriind rolul fiecăruia, principiile de implementare, precum și avantajele și limitările identificate. De asemenea, vor fi discutate posibile direcții de îmbunătățire și extensibilitate, evidențiind modul în care aceste subsisteme interacționează pentru a asigura funcționalitatea completă a nucleului.

3.1.1 Managerul memoriei fizice

Managerul de memorie fizică reprezintă o componentă esențială a nucleului, responsabilă pentru gestionarea paginilor de memorie disponibile. Așa cum am menționat în capitolul preliminar, arhitectura x86-64 împarte memoria în pagini (în mod obișnuit de 4 KiB, dar existând și pagini mai mari de 2 MiB sau 1 GiB). Rolul acestei componente este de a furniza o *interfață standardizată* prin care nucleul poate marca pagini ca fiind ocupate sau poate alocă pagini libere pentru utilizare ulterioară. Interfața tipică include funcții precum `lock_page`, `lock_pages`, `alloc_page`, `alloc_pages`, `free_page` și `free_pages`.

Din punct de vedere al implementării, bootloader-ul oferă nucleului, pe lângă încărcarea acestuia în memorie, o **hartă inițială a memoriei fizice**. Această hartă este prezentată sub forma unei liste de segmente, fiecare segment având o adresă de start, o dimensiune și un tip. Segmentele pot fi consecutive sau pot exista "găuri" între ele în spațiul de memorie. Tipul segmentului indică dacă memoria este liberă sau ocupată (de codul nucleului sau de dispozitive hardware).

Nucleul trebuie să mențină o *evidență precisă* a fiecărei pagini din memoria adresabilă, pentru a determina exact care sunt libere și care sunt ocupate. Pentru a realiza acest lucru, nucleul manipulează harta inițială pentru a genera o **reprezentare proprie a memoriei**, independentă de structura primită de la bootloader.

O metodă eficientă pentru a gestiona această evidență este utilizarea unui **bitmap**, în care fiecare bit reprezintă starea unei pagini (0 pentru liberă, 1 pentru ocupată), iar indexul bitului poate fi translatat la adresa fizică a paginii. Deoarece segmentele nu sunt întotdeauna consecutive, pot exista spații mari nealocate între segmente, ceea ce ar genera biți inutili dacă am alocă un bitmap global pentru întregul spațiu de memorie.

Pentru a evita acest lucru, nucleul grupează segmentele consecutive într-o *listă de segmente continue*, fiecare segment având un bitmap propriu, cu o bază și o dimensiune definite. Aceasta permite un control precis și eficient asupra memoriei disponibile. Inițializarea bitmap-urilor necesită încă harta oferită de bootloader, dar odată configurate, nucleul devine complet independent de aceasta.

Alocatorul caută bitul corespunzător paginii libere iterând prin lista de bitmap-uri și apoi prin biții efectivi ai fiecărui bitmap. Pentru alocarea unui pagini, se caută primul bit cu valoarea 0; pentru mai multe pagini consecutive, se caută un șir de n zerouri. Indexul bitului, combinat cu poziția bitmap-ului în listă, se traduce apoi într-o adresă fizică. Astfel, nucleul păstrează un control complet asupra fiecărei pagini.

Un aspect esențial al acestei abordări este faptul că structurile necesare pentru gestionarea memoriei ocupă și ele memorie. De aceea, nucleul folosește harta inițială pentru a estima necesarul de memorie pentru aceste structuri și apoi alocă un segment liber pentru stocarea lor. Putem calcula *overhead*-ul asociat prin raportul dintre numărul de pagini ocupate de structurile necesare subsistemului de memorie fizică și numărul total de pagini al sistemului.

3.1.2 Managerul memoriei virtuale

Managerul memoriei virtuale este o componentă care folosește paginile alocate de managerul memoriei fizice și este responsabilă pentru maparea și gestionarea tabelelor de pagini. Acesta realizează atât alocarea, cât și maparea paginilor, fie în contextul curent (dat de registrul CR3 care conține tabela **PML4**⁴), fie în contextul unui PML4 specific.

Există mai multe motive pentru care acest mecanism este esențial. Primul motiv este **izolarea proceselor** în propriul lor context de memorie: aceeași adresă virtuală poate corespunde unei adrese fizice diferite în funcție de proces, astfel încât un proces nu poate corupe memoria altuia. Accesul la pagini nealocate va genera un *page fault*⁵, protejând integritatea memoriei.

Un al doilea motiv este controlul **permisunilor la nivel de pagină**. Fiecare pagină poate fi marcată cu drepturi de acces specifice, precum *read*, *write*, *execute*, sau cu nivelul de privilegiu al procesorului necesar pentru acces. Această granularitate permite implementarea unor mecanisme de protecție, de exemplu împiedicarea unui proces să execute cod în zone care conțin date sau să modifice codul sursă al propriului program. De asemenea, sunt controlate relațiile dintre pagini și cache, cum ar fi *write-back* sau *write-through*, oferind un nivel suplimentar de optimizare și siguranță.

⁴**PML4** (*Page Map Level 4*) este nivelul superior al ierarhiei tabelelor de pagini în arhitectura x86-64, indicând baza tabelelor pentru traducerea adreselor virtuale în adrese fizice.

⁵Page fault este o excepție hardware care apare atunci când un proces încearcă să acceseze o pagină virtuală care nu este mapată în memoria fizică. Aceasta permite nucleului să gestioneze accesul și să aloce sau să recupereze pagina corespunzătoare.

Un al treilea motiv, deși nu este implementat în cadrul lucrării, constă în folosirea *page fault*-urilor pentru **swapping** pe disc⁶. În scenarii cu memorie limitată, paginile neutilizate pot fi stocate pe disc, iar când procesul solicită acele pagini, nucleul le încarcă în memorie, oferind procesului impresia că acestea au fost mereu disponibile. Acest mecanism permite extinderea efectivă a memoriei virtuale, fără ca aplicațiile să fie conștiente de lipsa temporară a anumitor pagini.

Din punct de vedere al implementării, managerul de memorie virtuală alocă adrese fizice prin intermediul alocatorului de memorie fizică și configurează tabelele de pagini pentru maparea acestora în spațiul virtual. Astfel, alocarea fizică nu se face niciodată direct de către alte componente, ci întotdeauna prin funcțiile oferite de managerul memoriei virtuale.

Pe lângă alocare și mapare, managerul memoriei virtuale este responsabil și pentru **schimbarea tabelelor de pagini curente**, funcționalitate esențială în timpul *context switch*-urilor. De asemenea, oferă funcții utilitare, precum translația de la o adresă virtuală la adresa fizică corespunzătoare sau conversia între adresele *user-space* și *kernel-space*, garantând astfel consistența și izolarea între diferitele contexte de execuție.

3.1.3 Heap-ul nucleului

Heap-ul reprezintă o zonă specializată de memorie destinată gestionării alocărilor de dimensiuni mici, mai mici decât o pagină de memorie⁷. Deși nucleul dispune de un alocator de memorie virtuală, acesta operează cu o granularitate fixă de 4 KiB, corespunzătoare dimensiunii unei pagini. În consecință, alocările de dimensiuni mici, cum ar fi structuri interne, tabele sau obiecte temporare, conduc la o utilizare inefficientă a memoriei.

Pentru a rezolva această problemă, nucleul include un alocator de tip **heap** cu o interfață standard, oferind funcțiile **kmalloc** pentru alocare și **kfree** pentru eliberare.

Heap-ul poate fi vizualizat ca o zonă de memorie divizată în segmente de dimensiuni variabile, fiecare segment având în față un *header*. Implementarea heap-ului se bazează pe o listă dublu-înlănțuită de aceste *header*-e, care conțin informații despre dimensiunea blocului și starea sa (ocupat sau liber). Deoarece majoritatea structurilor alocate necesită o aliniere maximă de 8 bytes, dimensiunile blocurilor libere sunt întotdeauna multipli de 8. Astfel, primul bit al câmpului de dimensiune este folosit pentru a semnaliza dacă blocul este liber sau ocupat, ceea ce permite o codare eficientă a stării blocului fără consum suplimentar de memorie.

Inițial, heap-ul este alocat prin alocatorul de memorie virtuală, rezervând un spațiu continuu de 2000 de pagini (aproximativ 8 MiB). Primul bloc al heap-ului este marcat ca liber și servește drept punct de plecare pentru alocările ulterioare.

Procesul de alocare parcurge lista de *header*-e și identifică primul bloc liber suficient de mare pentru a satisface cererea, incluzând dimensiunea *header*-ului și dimensiunea minimă de alocare. Dacă blocul este mai mare decât dimensiunea cerută, acesta este împărțit, iar un nou *header*

⁶Swapping este procesul prin care paginile neutilizate sunt mutate temporar pe disc pentru a elibera memoria fizică, fiind reîncărcate când sunt din nou necesare.

⁷Heap-ul este un mecanism standard de alocare dinamică a memoriei care permite rezervarea și eliberarea flexibilă a blocurilor de dimensiuni variabile în timpul execuției programului.

este inserat în listă pentru a marca segmentul nou creat. La eliberare, blocul verifică blocurile adiacente și, dacă acestea sunt libere, le combină, eliminând *header*-ele inutile. Acest mecanism reduce fragmentarea internă și permite alocări cu granularitate mai mică decât dimensiunea unei pagini.

Această implementare este minimală; în nucleele reale, alocatoarele de memorie sunt mult mai complexe și adaptate exact pentru dimensiunile structurilor utilizate, cum ar fi alocatoarele de tip *buddy*⁸ sau *slab allocators*⁹. Chiar și așa, interfața **kmalloc/kfree** rămâne generală, permițând înlocuirea alocatorului curent cu unul mai performant fără modificări majore ale nucleului.

3.1.4 Gestionarea întreruperilor

Gestionarea întreruperilor reprezintă un subsistem esențial al nucleului, responsabil pentru reacția sistemului la evenimente interne și externe. Nucleul definește structurile de date corespunzătoare arhitecturii x86-64; structura **IDT (Interrupt Descriptor Table)** și structura fiecărei intrări. Acest subsistem permite instalarea unui *handler* pentru fiecare vector de întrerupere, împreună cu parametrii asociați, cum ar fi nivelul de privilegiu și tipul de acces, prin configurarea corespunzătoare a intrării din tabelă.

Un element hardware crucial în acest mecanism îl constituie **PIC (8259 Programmable Interrupt Controller)**, un *controller* fizic montat pe placa de bază, care centralizează întreruperile generate de periferice și le transmite procesorului. Fiecare periferic activează o linie de întrerupere (*IRQ*) din PIC, iar controller-ul mapează aceste linii către vectorii corespunzători din IDT. PIC-ul gestionează 16 linii, fiecare putând fi mascată individual pentru controlul precis al fluxului de întreruperi.

În configurația standard, IDT conține 256 de intrări, dintre care primele 32 sunt rezervate excepțiilor procesorului x86-64, care nu pot fi modificate sau dezactivate. Întreruperile hardware inițiale generate de periferice se suprapun cu aceste intrări rezervate, generând un conflict. Pentru rezolvarea acestuia, nucleul remapează liniile IRQ0-15 ale PIC-ului pe vectorii 32-47, prin scrierea offsetului în registrele I/O corespunzătoare și mascare temporară a tuturor liniilor, prevenind astfel generarea de întreruperi necontrolate în timpul inițializării.

Deși PIC-ul este o tehnologie veche, menținută în calculatoare moderne pentru compatibilitate, nucleele actuale care suportă SMP folosesc **APIC (Advanced Programmable Interrupt Controller)** pentru controlul precis al întreruperilor pe sisteme multi-core. În contextul lucrării, PIC-ul este suficient pentru gestionarea întreruperilor. Astfel, pentru exemplificare, sunt utilizate doar două întreruperi hardware: vectorul 33 pentru tastatura PS/2 și vectorul 32 pentru timer-ul PIT, despre care se va discuta în detaliu în secțiunea următoare.

⁸Buddy allocator este un algoritm de alocare a memoriei care împarte blocurile de memorie în puteri ale lui doi și permite combinarea lor pentru a satisface cerințele de dimensiune.

⁹Slab allocator este un alocator de memorie care păstrează cache-uri de obiecte de dimensiuni fixe pentru a reduce fragmentarea și a crește performanța alocării repetate a aceluiași tip de obiect.

3.1.5 Mecanisme de temporizare

La baza funcționării nucleului se află un **timer periodic**, care permite contabilizarea timpului și oferă nucleului control periodic asupra execuției. Implementarea acestei funcționalități necesită resurse hardware dedicate; pe platforma x86-64 există mai multe opțiuni, printre care **PIT (Programmable Interval Timer)**, **HPET (High Precision Event Timer)** sau **APIC timer**.

Principiul de funcționare este similar pentru toate aceste timere: nucleul configurează un interval, iar timerul generează întreruperi periodice către procesor la fiecare interval stabilit. Intervalul tipic este de câteva milisecunde. În implementarea prezentată, s-a ales **PIT-ul**, deoarece este simplu de configurat și oferă o rezoluție suficientă de aproximativ 10 ms. Alte timere precum **HPET** sau **APIC timer** oferă o precizie mai bună și frecvențe mai înalte.

Configurarea PIT-ului implică scrierea unor valori specifice în porturile de I/O dedicate, pentru a seta frecvența dorită. Așa cum este menționat anterior, întreruperile hardware sunt inițial mascate de PIC, astfel că timerul configurat nu va întrerupe imediat nucleul. Dezactivarea măștii (*unmasking*) se face ulterior, atunci când planificatorul este pregătit să gestioneze întreruperile periodice și să preia controlul asupra execuției.

3.1.6 Planificatorul și comutare de context

Planificatorul (*scheduler*) este componenta nucleului responsabilă de alocarea timpului procesorului între toate task-urile active. Rolul său principal este de a decide dacă un *task* a rulat suficient timp și de a determina care task va fi executat în continuare.

Nucleul implementat în această lucrare suportă **preemptive multitasking**¹⁰, permițând rularea aparent paralelă a mai multor procese chiar și pe un singur nucleu fizic. Paralelismul este doar o iluzie: în realitate, fiecare proces sau thread de nucleu rulează secvențial pentru o perioadă foarte scurtă de timp, determinată de temporizatorul periodic (PIT), setat la 10 ms. La fiecare întrerupere, nucleul decide care task va fi rulat și efectuează un *context switch*.

Algoritmul de planificare utilizat în acest proiect este **Round-Robin**. Nucleul menține o listă dublu înălțuită ciclică (*ready queue*) a task-urilor pregătite și un pointer către task-ul curent. La fiecare întrerupere periodică, schedulerul selectează următorul task din listă. În cazul în care lista de task-uri pregătite este goală, nucleul rulează un task special, *idle*, care nu efectuează operații utile și menține procesorul într-o stare cu consum redus de resurse, deoarece procesorul trebuie să execute întotdeauna ceva și nu poate fi oprit complet.

¹⁰Preemptive multitasking înseamnă că nucleul poate întrerupe un task activ pentru a permite altuia să ruleze, asigurând utilizarea echitabilă a procesorului între toate task-urile active.

Fiecare task este reprezentat de o structură complexă **Task**, care conține diverse câmpuri precum: identificatorul task-ului, referințe către task-ul părinte și copii, referințe către diferite liste (lista globală a task-urilor, lista task-urilor gata de execuție), starea task-ului (*gata de execuție, terminat, în așteptare*), contextul procesorului, contextul memoriei (adresa pentru stiva utilizator, dacă este cazul, și tabela de pagini corespunzătoare), contextul sistemului de fișiere (directorul curent) și tabela de descriptori de fișiere.

Mecanismul de comutare de context

Comutarea de context (*context switch*) este realizată complet în software de către nucleu și presupune salvarea stării procesorului pentru task-ul curent și încărcarea stării următorului task. Pașii principali sunt următorii:

1. Întreruperea periodică și tranziția de privilegii

Temporizatorul periodic generează o întrerupere. Dacă task-ul curent rulează în spațiu utilizator (**ring 3**), procesorul trece în **ring 0** folosind stiva de nucleu configurată în TSS¹¹. Această tranziție este atomică și asigură un mediu sigur pentru execuția codului de nucleu.

2. Salvarea contextului task-ului curent

Rutina de întrerupere software încarcă manual pe stiva de nucleu o structură **CPU**, care include: **RAX**, **RBX**, **RCX**, **RDY**, **RBP**, **RSI**, **RDI**, **R8-R15** precum și **RIP**, **RFLAGS**, **CS**, **SS**. În plus, vârful stivei de nucleu este salvat în structura task-ului curent. Până aici codul rulat este scris în assembly, iar de aici procesorul continuă să execute rutina **schedule** scrisă în C++, care reprezintă planificatorul propriu-zis.

3. Alegerea următorului task

În C++, planificatorul selectează următorul task din *ready queue* în mod Round-Robin. Dacă task-ul următor este *user-space*, nucleul va schimba spațiul de memorie corespunzător utilizând managerul memoriei virtuale. Registrul **CR3** este actualizat cu **PML4**-ul task-ului următor, realizând comutarea completă a spațiului de memorie. Thread-urile de nucleu nu necesită schimbarea **CR3**, deoarece toate rulează deja în ring 0 și au mapat nucleul permanent. În plus, referința globală la task-ul curent este actualizată cu task-ul selectat.

4. Ieșirea din întrerupere

La revenirea la codul în assembly, procesorul își schimbă stiva curentă cu stiva stocată în task-ul selectat. Pe noua stivă, procesorul preia contextul stocat în structura **CPU** și reia execuția task-ului comutat. În acest punct, contextul a fost schimbat, iar noul proces își continuă execuția până la următoarea întrerupere.

¹¹TSS conține adresa stivei de nucleu pentru fiecare nivel de privilegii. Aceasta permite nucleului să ruleze sigur în ring 0 chiar dacă procesul curent era în ring 3.

Acest mecanism se repetă permanent pentru toate task-urile din *ready queue*, realizând multitasking preemptiv complet controlat de software. Nucleul face abstracție între procesele *user-space* și thread-urile de nucleu prin conceptul de **Task**, deși există următoarele diferențe:

- **Procesele utilizator** – rulează în ring 3, au o stivă utilizator în plus față de stiva de nucleu (care este comună ambelor tipuri de **Task**) și au un context propriu de memorie.
- **Thread-urile de nucleu** – rulează în ring 0 și nu au un context propriu de memorie, deoarece memoria nucleului este mapată permanent în orice **Task**.

3.1.7 Subsistemul de input

Pentru ca utilizatorul să poată interacționa cu sistemul de operare, nucleul trebuie să ofere un mecanism de interceptare și procesare a evenimentelor de intrare. Cel mai simplu model de interacțiune este prin intermediul tastaturii, într-un stil de interfață de tip **CLI** (*Command Line Interface*). Evident, un sistem de operare complet ar trebui să suporte și alte periferice de intrare, precum mouse-ul sau dispozitive USB, însă în cadrul acestui nucleu s-a ales implementarea unui driver simplu pentru tastatura **PS/2**.

Funcționarea driverului este bazată pe **întreruperi** generate de tastatură. La fiecare apăsare sau eliberare a unei taste, tastatura trimite o întrerupere procesorului, asociată vectorului 33 din *IDT*¹², iar nucleul citește un octet de la portul de I/O 0x60. Acesta reprezintă **scancode-ul** corespunzător evenimentului deși, în anumite cazuri, scancode-urile sunt formate din mai mulți octeți.

Pentru a evita procesarea directă în contextul întreruperii, nucleul trimite aceste scancode-uri către un *thread* dedicat, responsabil de gestionarea inputului. Thread-ul dispune de un buffer intern implementat sub forma unei cozi. La fiecare întrerupere, scancode-ul citit este introdus în coadă, iar thread-ul, care se află de cele mai multe ori suspendat, este trezit și procesează evenimentele în mod asincron. Thread-ul dedicat tastaturii se ocupă și de inițializarea unor parametri specifici controller-ului PS/2, precum rata de repetare a caracterelor atunci când o tastă este menținută apăsată.

Procesarea constă în traducerea scancode-urilor în caractere ASCII, ținând cont de starea tastelor speciale, precum **Shift**, **Caps Lock** etc. Ulterior, caracterele rezultate sunt trimise către *terminalul virtual* (*framebuffer terminal*). Tot la acest nivel sunt interpretate și combinațiile speciale de taste, precum **Ctrl+C**, **Ctrl+D**, **Ctrl+U** sau **Ctrl+L**, care permit interacțiunea de bază cu procesele și cu shell-ul.

Această componentă nu este încă foarte dezvoltată și poate fi considerată mai degrabă un mecanism minimal, menit să ofere utilizatorului posibilitatea de a interacționa cu sistemul. O implementare mai avansată ar introduce o abstracție suplimentară peste scancode-uri, similar conceptului de *keycodes* din Linux, permițând astfel maparea uniformă a mai multor tipuri de tastaturi la nivelul nucleului.

¹²IDT - *Interrupt Descriptor Table* - tabelul de întreruperi folosit de procesor pentru a direcționa execuția către rutina corespunzătoare unei întreruperi.

De asemenea, un nucleu modern ar trebui să includă suport pentru tastaturi USB. În cadrul acestui proiect, pentru rularea sub emulatori precum QEMU, Bochs sau VirtualBox, suportul pentru PS/2 este suficient. Într-un mediu fizic, folosirea acestui driver presupune existența unei tastaturi PS/2, ceea ce poate fi o limitare, deoarece aceste dispozitive nu mai sunt foarte răspândite.

3.1.8 Terminalul virtual

Pentru a reacționa la input-ul utilizatorului, nucleul are nevoie de un mecanism simplu de afișare a textului pe ecran. Cel mai natural model este cel al unui **terminal**, care stă la baza interfețelor *CLI*, menționate mai sus. Acest subsistem este extrem de important în dezvoltarea nucleului, deoarece permite afișarea de mesaje utile și depanarea altor componente încă din fazele incipiente.

O soluție clasică și foarte simplă pentru output este folosirea portului serial, prin care caracterele sunt trimise către un terminal extern. Totuși, în acest proiect s-a ales ca output-ul să fie direct pe monitor, pentru a putea rula sistemul și pe hardware real, nu doar în emulator.

În trecut, afișarea textului se realiza prin **VGA text mode**, unde hardware-ul putea scrie direct text pe ecran, împreună cu diferite atribute de culoare. Textul era stocat într-un buffer la o adresă fixă (0xB8000), bine stabilită. Acest mecanism era rapid și simplu, însă astăzi este considerat *legacy* și nu mai este disponibil în modul modern UEFI.

În locul lui, nucleul primește de la bootloader o structură ce descrie **framebuffer-ul**, adică zona de memorie dedicată pixelilor de pe ecran, dimensiunile acestuia (lățime și înălțime în pixeli) și formatul pixelilor (de exemplu, *ARGB*). Pe baza acestor informații, nucleul emulează un mod *text mode*, desenând caractere în software. Fontul¹³ este stocat în memoria nucleului sub formă de bitmap generat extern. Astfel, nucleul oferă o interfață prin care se poate scrie text pe ecran.

Terminalul implementat nu se rezumă doar la afișarea caracterelor, ci oferă și suport pentru **ANSI escape sequences**¹⁴. Aceste secvențe sunt interpretate similar unor funcții. Inspirat de conceptele UNIX, mecanismul de scriere pe ecran este expus și în *user-space* ca un fișier, astfel încât programele pot deschide acest fișier și scrie textul, care va fi afișat pe ecran. Implicit, citirea este realizată cu *line buffering*.

O altă funcționalitate esențială este **scroll-ul**. În timp ce afișarea caracterelor presupune doar scriere în framebuffer, scroll-ul necesită și citire pentru a muta liniile cu un rând mai sus. Această operație se comportă diferit în emulator și pe hardware real: în emulator, citirea din framebuffer este aproape instantanee, însă pe un PC real poate fi extrem de lentă, până la câteva secunde pentru curățarea completă a ecranului.

¹³Fontul utilizat în nucleu este **SF Mono** dezvoltat de Apple, folosit exclusiv în scop demonstrativ. Mai multe detalii pot fi găsite pe site-ul Apple: <https://developer.apple.com/fonts/>.

¹⁴ANSI escape sequences sunt secvențe speciale de caractere care permit controlul culorii textului, poziția cursorului și alte efecte de control.

Pentru a rezolva această problemă, nucleul folosește un **backbuffer**: o zonă de memorie în care se realizează desenarea completă, iar rezultatul este ulterior copiat în framebuffer-ul video. Această tehnică, cunoscută ca *double buffering*, elimină necesitatea citirilor lente din VRAM și asigură performanță acceptabilă atât în emulator, cât și pe hardware real.

Terminalul implementat nu este complet compatibil cu un terminal standard (precum *VT100*), însă suportă subsetul cel mai utilizat de funcționalități: culori de bază, mutarea cursorului și combinații speciale de taste.

3.1.9 Sistemul virtual de fișiere

Sistemul de fișiere virtual (VFS) reprezintă unul dintre cele mai importante mecanisme ale nucleului, deoarece permite proceselor să acceseze într-un mod unitar resurse foarte diferite. Din perspectiva unui proces, aproape orice interacțiune cu lumea externă se face prin fișiere: fie că este vorba despre un executabil, date salvate pe disc, un *pipe* pentru comunicarea între procese sau chiar un dispozitiv expus în stil UNIX ca fișier. Fără un astfel de strat de abstractizare, fiecare aplicație ar trebui să știe singură cum să comunice direct cu discul sau cu formatele concrete de stocare, lucru imposibil de gestionat și extrem de vulnerabil din punct de vedere al securității.

VFS rezolvă această problemă oferind o interfață unică și abstractă. Indiferent dacă datele provin dintr-un fișier pe disc, dintr-un sistem în memorie sau dintr-un canal de comunicație, procesul folosește aceleași apeluri simple: **open**, **read**, **write**, **close**.

Implementarea VFS presupune utilizarea unui set de structuri și tabele, organizate pe nivele de abstractizare:

- **FilesystemType** – descrie un tip de sistem de fișiere (ex. **ext2**, **ramfs**) și operațiile sale asociate. Nucleul menține o listă cu toate tipurile de sisteme de fișiere înregistrate.
- **Superblock** – reprezintă o instanță concretă a unui sistem de fișiere montată pe un anumit dispozitiv (ex. un hard disk, sau doar memoria RAM pentru **ramfs**). Fiecare superblock are propriul director rădăcină (*root inode*).
- **Mount** – leagă o instanță de sistem de fișiere de un punct de montare din arborele global. Toate sistemele montate se integrează într-o singură ierarhie, cu rădăcina **/**.
- **Inode** – structura din nucleu care reprezintă metadatele unui fișier (dimensiune, tip, permisiuni, etc.) și operațiile asociate (ex. **lookup**, **mkdir**, **link**, **unlink**).
- **File** – cea mai abstractă structură, prin care un proces interacționează cu un obiect. Poate face referire la un *inode*, la un *pipe* sau la un dispozitiv. Nucleul menține o tabelă globală de fișiere, iar fiecare proces are o tabelă proprie cu descriptorii săi.

Fluxul unui apel sistem tipic, precum `open("dir/file", ...)`, este următorul:

1. Procesul efectuează apelul de sistem `open` cu o cale relativă la directorul său curent (*current working directory*), stocat în structura `Task`.
2. Nucleul pornește din *inode*-ul directorului curent și, pentru fiecare componentă a căii, apelează operația `lookup`, obținând *inode*-ul următor.
3. Dacă fișierul este găsit, kernelul alocă o structură `File` în tabela globală și inserează o referință la ea în tabela de fișiere a procesului.
4. Apelul sistem returnează poziția intrării (*descriptorul de fișier*) care poate fi folosit ulterior de proces pentru alte apeluri precum `read`, `write` sau `close`.

Pentru performanță, nucleul menține un *cache de inode-uri*: modificările se fac asupra structurilor din memorie, iar actualizarea pe disc are loc doar atunci când *inode*-ul nu mai este folosit de niciun proces. Extensii moderne, precum *dentry cache* din Linux, optimizează suplimentar rezolvarea căilor. În implementarea actuală, nucleul suportă două tipuri de sisteme de fișiere:

- **ramfs** – funcțional în permanență, montat implicit la `/` (radăcina arboreșcenței de fișiere), folositor pentru rularea fără disc.
- **EXT2** – implementare parțială, doar citire, bazată pe un driver simplu pentru discuri ATA. Funcționează în emulator (QEMU), dar nu pe hardware modern care folosește controlere SATA sau NVMe.

Deși minimală, această implementare este suficientă pentru a face nucleul utilizabil. Există inclusiv suport pentru *symlink*-uri, ceea ce face ca arborele de fișiere să fie mai flexibil. În viitor, nucleul poate fi extins cu sistem de *caching* mai avansat, suport pentru scriere pe EXT2 și drivere pentru dispozitive de stocare moderne.

3.2 Inițializarea nucleului și lansarea primului proces

Odată ce nucleul este încărcat în memorie de către bootloader, acesta apelează funcția de intrare `kmain`, de unde începe execuția codului de inițializare. Procesul complet de pornire se desfășoară în mai multe etape:

1. **Activarea instrucțiunilor SSE.** Nucleul activează instrucțiunile SSE, însă nu le utilizează efectiv în execuția curentă. Motivul este că includerea lor în mecanismul de schimbare de context ar face operația mai costisitoare, deoarece ar trebui salvați și restaurați mai mulți regiștri. Activarea se face mai mult pentru compatibilitate și pentru eventuale utilizări viitoare.
2. **Inițializarea terminalului virtual.** Kernelul configurează un terminal simplu ce scrie direct în *framebuffer*. În această etapă nu se folosește încă double buffering, deoarece nu există un alocator de memorie disponibil. Se face această alegere pentru a putea afișa mesaje cât mai devreme, lucru esențial pentru depanare și urmărirea stadiilor inițiale de boot.

3. **Inițializarea managerului de memorie fizică.** Bootloaderul furnizează harta memoriei fizice, pe baza căreia kernelul marchează zonele libere și pe cele rezervate (de exemplu pentru BIOS, dispozitive, sau nucleul însuși). Astfel, nucleul are acum un mecanism centralizat prin care poate aloca și elibera cadre de memorie fizică.
4. **Inițializarea managerului de memorie virtuală.** După configurarea memoriei fizice, kernelul activează paginația și mecanismele de mapare virtuală. Din acest moment, nucleul poate proteja spațiile de memorie ale proceselor și poate izola *user-space* de *kernel-space*. Tot acum se alocă spațiu pentru backbuffer-ul terminalului, ceea ce permite activarea double buffering-ului și obținerea unei afișări mai fluide.
5. **Configurarea structurilor procesorului (IDT, GDT, syscall).** Urmează setarea tabelor critice pentru execuția sistemului:
 - **IDT** – tabela de întreruperi, necesară pentru a răspunde evenimentelor hardware și software.
 - **GDT** – tabela de segmente, pentru definirea spațiului de adresare și a nivelurilor de privilegii.
 - Instrucțiunea **syscall** – activată pentru a permite proceselor *user-space* să invoce apeluri de sistem într-un mod modern și rapid.
6. **Inițializarea heap-ului.** După ce memoria virtuală este disponibilă, kernelul configurează și heap-ul său intern. Acest heap permite alocarea obiectelor de dimensiuni mici și flexibile (precum structurile pentru tabele sau liste interne). Este un pas esențial pentru a putea construi componentele de nivel superior.
7. **Înregistrarea și montarea sistemelor de fișiere.** Nucleul înregistrează cele două sisteme de fișiere suportate: **ramfs** și **ext2**. Apoi, **ramfs** este montat ca root. În această fază există un root, dar el este complet gol.
8. **Popularea directorului rădăcina.** Kernelul creează manual directoare de bază precum **/dev**, **/bin**, **/mnt**, etc. De asemenea, sunt inițializate dispozitive virtuale precum **/dev/fb**, **/dev/fbterm**, **/dev/serial**, și sunt create executabile de bază incluse direct în binarul kernelului (de exemplu un shell simplu).
9. **Montarea unei imagini ext2.** Kernelul încearcă să monteze o imagine **ext2** în directorul **/mnt**. Această operație funcționează doar în emulator (QEMU).
10. **Inițializarea planificatorului.** Schedulerul este acum activat, iar firul de execuție curent (care rula **kmain**) este introdus în coada de task-uri *ready*, transformându-se într-un task normal. Din acest punct, execuția nu mai este secvențială, ci este gestionată de mecanismul de multitasking.
11. **Crearea thread-ului de input al nucleului.** Kernelul creează și pornește un fir de execuție dedicat inputului (pentru tastatura). Acest thread rulează concurent și se ocupă exclusiv de gestionarea evenimentelor de intrare.

12. **Lansarea shell-ului.** Nucleul încearcă să lanseze un shell: mai întâi caută un binar mai avansat în `/mnt/bin/sh`, iar dacă acesta nu este disponibil, revine la shell-ul simplu inclus în nucleu (`/bin/sh`). Acest shell este pornit ca un proces *user-space*.
13. **Terminarea task-ului de inițializare.** După ce shell-ul este lansat și thread-ul de input rulează, taskul `kmain` își încheie execuția prin apelul `exit`. În acest moment, sistemul rămâne cu două taskuri principale: shell-ul și thread-ul de input al nucleului.

Astfel, nucleul este complet inițializat, iar de acum înainte interacțiunile reale se realizează prin programele *user-space*.

Capitolul 4

Spațiul utilizator

Spațiul utilizator reprezintă ansamblul de programe care rulează în ring 3, separate de nucleu pentru a asigura securitate și stabilitate. După cum am menționat și în capitolul introductiv, nucleul și spațiul utilizator comunică printr-o interfață bine definită. În cadrul acestui capitol se vor analiza mecanismele fundamentale prin care se realizează această interacțiune, precum și modul în care programele utilizator sunt create, executate sau eventual portate.

4.1 Apelurile de sistem

Apelurile de sistem (*system calls*) reprezintă interfața fundamentală prin care programele din spațiul utilizator accesează funcționalități ale nucleului.¹

După cum a fost descris în capitolul anterior, **nucleul** este alcătuit din multiple subsisteme, fiecare responsabil de anumite funcționalități. Acestea sunt expuse spațiului utilizator exclusiv prin intermediul apelurilor de sistem. De exemplu, un program care dorește să scrie într-un fișier nu poate accesa direct discul, ci solicită nucleului să efectueze această operație prin apelul de sistem **write**.

Nucleul oferă o **tabelă extensivă de apeluri de sistem**. Multe dintre acestea au origini istorice în UNIX, dar există și apeluri specifice sistemelor moderne, precum Linux (de exemplu **getdents**²). Din punct de vedere *low-level*, apelurile de sistem se aseamănă cu apelurile de funcții. Diferența este că registrul **RAX** stochează identificatorul apelului (**write**, **execve**, **fork** etc.), iar transferul de control se face prin instrucțiunea **syscall**.³

Această instrucțiune, odată executată, realizează tranziția procesului din **ring 3** în **ring 0**, permițând nucleului să preia controlul și să execute operația solicitată.

În practică, programele nu folosesc direct instrucțiunea **syscall**, ci utilizează funcții oferite de biblioteci standard, precum **libc**. Aceste funcții acționează ca un *wrapper*, pregătind parametrii necesari și invocând apelul de sistem corespunzător. Astfel, programatorii au la dispoziție o interfață mai accesibilă și mai portabilă, în timp ce detaliile de implementare rămân ascunse în spatele acestui strat intermediar.

¹Apelurile de sistem constituie singura cale de acces la funcțiile interne ale nucleului.

²Apelul **getdents** este utilizat pentru citirea intrărilor dintr-un director.

³În trecut, apelurile de sistem erau invocate prin întreruperi software, de exemplu **int 0x80** în Linux.

4.2 Crearea și execuția programelor

În această secțiune se descrie procesul prin care un cod sursă C poate fi transformat într-un program capabil să ruleze efectiv folosind nucleul implementat în cadrul acestui proiect. Procesul presupune utilizarea unor instrumente standard pentru limbajul C și generarea unui executabil **ELF64** [5] static (fără *linking* dinamic).

La nivel de bază, este posibil să se scrie cod în assembly, invocând direct apeluri de sistem, însă această abordare este dificilă și nepractică. Ideal este să se utilizeze funcția standard `main(argc, argv)`, nucleul pregătind deja argumentele pentru `main` pe stiva utilizator în momentul apelului `execve`.

Toate apelurile de sistem, împreună cu diverse flaguri și structuri, sunt definite într-un header numit `sys.h`. Având în vedere că **System V AMD64 ABI** [6] corespunde aproape exact modului de transmitere a parametrilor pentru apelurile de sistem (cu excepția registrului `RCX`), compilatorul C poate plasa argumentele în registrii corespunzători, urmând ca implementarea să pună identificatorul apelului în `RAX` și să execute instrucțiunea `syscall`. Aceasta reprezintă funcționalitatea fișierului `sys.asm`, care implementează definițiile din `sys.h` pentru fiecare apel de sistem.

Pe lângă implementarea apelurilor de sistem, `sys.asm` furnizează și un mecanism similar unui `crt0`⁴ pentru a oferi interfața standard `main(argc, argv)`. În mod implicit, linkerul urmărește simbolul `_start` ca punct de intrare al executabilului, și nu simbolul `main`. Prin urmare, `sys.asm` definește simbolul `_start`, care preia argumentele de pe stiva pregătită de nucleu, apelează simbolul global `main` și, la final, invocă apelul de sistem `exit` cu codul de retur al funcției `main`. Fișierul `sys.asm` este asamblat cu **NASM**⁵ pentru a produce obiectul `sys.o` necesar construirii executabilului.

Astfel, un program de bază necesită doar două fișiere: `sys.h` și `sys.asm`, împreună cu fișierul sursă `main.c`. Compilarea se realizează pe un sistem gazdă compatibil x86-64 și generează un executabil ELF64 static, fără a implica nucleul descris în lucrare. O posibilă comandă de compilare este următoarea:

```
gcc main.c sys.o -static -nostdlib
```

Pentru a rula executabilul, acesta trebuie fie mutat pe o imagine de disc formatată cu EXT2, fie încorporat direct în memoria nucleului, urmând ca mai apoi să fie expus prin *ramfs*.

Folosind header-ul `sys.h` se pot construi biblioteci suplimentare de tip **libc** pentru a simplifica scrierea aplicațiilor în spațiul utilizator. În cadrul acestei lucrări, am implementat un `user.h` minimal, care adaugă câteva funcții utile din `ctype.h` și `string.h`, precum și o clonă a `printf`. Astfel, o parte din limitările sistemului nu se datorează nucleului, ci caracterului minimal al spațiului utilizator, dezvoltarea unui mediu complet nefiind obiectivul acestei lucrări.

⁴Un `crt0` (*C runtime zero*) reprezintă un cod minimalist care inițializează un program C înainte de apelul funcției `main`.

⁵**NASM** (*Netwide Assembler*) este un assembler open-source pentru arhitectura x86-64. Informații oficiale și documentație pot fi găsite la www.nasm.us.

4.3 Ecosistemul minimal de programe

Așa cum s-a menționat anterior, nucleul constituie fundamentul unui sistem de operare, dar în mod izolat nu oferă funcționalități suficiente pentru o utilizare reală. Pentru a evidenția capabilitățile nucleului dezvoltat, a fost realizat un **set minimal de programe în spațiul utilizator**, menite să ilustreze apelurile de sistem și interacțiunea dintre componentele de bază.

Un prim element esențial a fost implementarea unui **shell**. În sistemele UNIX, shell-ul reprezintă interfața dintre utilizator și nucleu, având rolul de a interpreta comenzile și de a lansa procese. Shell-ul dezvoltat este unul minimalist, fără suport pentru variabile, istoric sau editare avansată a liniei de comandă. Totuși, acesta permite execuția de programe, redirecționări și *pipe*-uri, suficiente pentru a valida mecanismele fundamentale implementate în nucleu. De asemenea, este și primul program lansat după inițializarea sistemului.

Pentru testare au fost dezvoltate și **utilitare inspirate din UNIX**, precum `ls`, `echo`, `cat` sau `grep`. Acestea nu urmăresc conformitatea POSIX⁶, ci doar reproducerea funcționalităților elementare pentru verificarea apelurilor de sistem fundamentale (`open`, `read`, `write`, `close`).

Pentru obiectivul de *self-hosting*, a fost portat un **editor de text**, bazat pe un proiect anterior și adaptat la apelurile de sistem actuale. Acesta permite editarea fișierelor text și se bazează pe suportul pentru *ANSI escape codes* implementat în terminalul virtual, oferind astfel o interfață vizuală simplă.

Etapa cea mai complexă a fost integrarea unui **compilator C**. GCC, utilizat pentru construirea nucleului, este prea voluminos pentru scopul acestei lucrări, astfel că a fost ales **TCC (Tiny C Compiler)** [7]. Procesul de portare a presupus compilarea statică pe un sistem Linux x86-64, transferul pe o imagine de disc EXT2 și analiza apelurilor de sistem invocate. Apeluri precum `getuid`, neesențiale într-un nucleu fără suport multi-utilizator, au primit implementări minimale. De asemenea, apelul `exit_group`, utilizat în Linux pentru terminarea simultană a tuturor firelor unui proces, a fost înlocuit cu `exit`, întrucât nucleul dezvoltat nu oferă suport pentru fire de execuție la nivel de utilizator. În final, toate apelurile necesare au fost integrate, iar TCC a devenit funcțional, permițând compilarea directă a programelor C.

Astfel, sistemul obținut include un nucleu funcțional și un spațiu utilizator cu un shell, utilitare, editor de text și compilator C. Această combinație permite extinderea sistemului din interior, prin dezvoltarea de programe suplimentare (de exemplu, `wc`⁷, sau un shell mai avansat). Pentru suportul procesului de compilare sunt incluse și fișiere predefinite: `sys.h` și `sys.o`, organizate astfel: `/bin` pentru executabile, `/include` pentru fișiere header, iar `/lib` pentru obiecte precompilate. Imaginea de disc este montată de nucleu în directorul `/mnt` prin intermediul sistemului de fișiere virtual.

Prin aceste componente, sistemul demonstrează maturitatea arhitecturii implementate, chiar dacă există limitări: driverul EXT2 suportă doar citire (*read-only*), iar *ramfs* este volatil. Chiar și în aceste condiții, mediul obținut este suficient pentru a valida funcționalitatea nucleului și pentru a permite dezvoltarea internă de aplicații.

⁶POSIX reprezintă standardul care definește interfețele și comportamentele portabile ale sistemelor de operare.

⁷`wc` (word count) – program care numără linii, cuvinte și caractere dintr-un fișier.

Capitolul 5

Concluzii

5.1 Rezultate obținute

Lucrarea de față a avut ca obiectiv dezvoltarea unui nucleu minimal pentru arhitectura x86-64, în modul *long mode* (64-bit), urmărind atât aprofundarea mecanismelor fundamentale ale sistemelor de operare, cât și obținerea unei experiențe practice prin implementarea lor.

Nucleul implementat include un **Virtual File System (VFS)**, care oferă o interfață unitară pentru fișiere și suportă atât un *ramfs* simplu, cât și **EXT2** în mod *read-only*. Multitasking-ul este realizat pe un singur nucleu, folosind un scheduler *round-robin*, cu suport pentru *kernel threads* și procese utilizator, asigurând separare completă a memoriei și tranziție între *ring-uri*. Pentru comunicarea între procese există suport pentru *pipe-uri*.

În spațiul utilizator, a fost implementat un **ecosistem software minimal**, compus dintr-un shell, un editor de text minimalist și utilitare de bază (*ls*, *cat*, *echo*). Acestea nu reproduc în totalitate standardele POSIX, ci oferă funcționalitatea minimă necesară pentru demonstrarea interacțiunii cu nucleul. De asemenea, portarea compilatorului **TCC** permite compilarea directă a programelor C pe sistem, realizând un nivel esențial de *self-hosting*.

Substratul de I/O este asigurat prin drivere pentru controller-ul ATA, tastatura PS/2 și portul serial, toate expuse printr-o interfață unitară de fișiere, în spiritul sistemelor UNIX. Terminalul este emulat software folosind *framebuffer*-ul furnizat de bootloader. Sistemul rulează atât în emulator (QEMU), cât și pe hardware modern x86-64, confirmând compatibilitatea cu platforme reale.

5.2 Limitările nucleului

Una dintre cele mai semnificative limitări ale nucleului este legată de mecanismele de comunicare între procese (*IPC*). Nucleul nu implementează semnale tipice UNIX și nu oferă nicio interfață compatibilă cu standardul BSD Socket API, ceea ce înseamnă că nici comunicarea prin *Unix Domain Sockets* nu este posibilă. Această situație limitează semnificativ dezvoltarea aplicațiilor care necesită interacțiuni complexe între procese.

Pe partea de temporizare, nucleul nu furnizează apeluri de sistem precum *sleep*, *alarm* sau *gettimeofday*, astfel încât programele din spațiul utilizator nu beneficiază de niciun me-

canism pentru măsurarea sau controlul timpului. Aceasta restricție împiedică implementarea funcționalităților care depind de temporizare precisă sau programarea sarcinilor periodice.

De asemenea, nucleul este conceput pentru un singur nucleu și nu suportă SMP (*Symmetric Multiprocessing*). În consecință, nu există driver pentru APIC, care ar fi permis o gestionare mai modernă a întreruperilor și temporizarea în medii multicore. În plus, nucleul nu implementează thread-uri în spațiul utilizator, existând doar *kernel threads*. Totuși, arhitectura actuală a nucleului oferă majoritatea mecanismelor necesare pentru a permite implementarea facilă a thread-urilor utilizator în versiuni viitoare.

La nivel de suport hardware, nucleul nu include drivere pentru periferice moderne, precum **USB 3.x** (xHCI), sau pentru controlere de disc complexe, cum ar fi **SATA** (AHCI) sau **NVMe**, limitând astfel capacitatea sistemului de a interacționa cu echipamente contemporane.

Totodată, sistemul nu gestionează utilizatori și privilegii, iar aspectele de securitate sunt minimale. De asemenea, subsistemele nucleului pot fi supuse unor optimizări, în special alocatoarele de memorie, pentru a crește eficiența și stabilitatea generală a sistemului.

5.3 Direcții viitoare

Pe baza experienței acumulate în această lucrare, mi-ar plăcea să explorez dezvoltarea unui nucleu nou, de data aceasta în **Rust**, un limbaj low-level modern ce combină controlul sigur al memoriei cu zero-cost abstractions, potrivit pentru dezvoltarea unui sistem de operare.

Aș dori ca, încă de la început, nucleul să suporte **SMP** și thread-uri în spațiul utilizator, astfel încât aplicațiile să poată rula simultan pe mai multe nuclee, profitând de paralelismul real oferit de hardware.

Arhitectura ar fi mai apropiată de un **microkernel**, cu cât mai multe componente mutate în userspace. De exemplu, terminalul virtual ar putea fi pur și simplu un program userspace, fără privilegii kernel, crescând modularitatea și siguranța.

Inspirat de **Plan 9 from Bell Labs**¹, aș dori ca multe funcționalități, inclusiv comunicația între procese, să fie realizate prin sistemul de fișiere, prin fișiere sintetice sau *file servers*, astfel încât IPC-ul să fie integrat într-un model unitar și uniform.

De asemenea, mi-ar plăcea să definesc un set de apeluri de sistem mai ortogonal și intuitiv, cu nume care reflectă clar operațiile efective, evitând ambiguitățile întâlnite în UNIX sau POSIX. În acest fel, nucleul ar deveni un cadru flexibil pentru experimentarea ideilor proprii, fără a imita strict sistemele existente.

¹Plan 9, dezvoltat de echipa care a creat UNIX, propune un model în care toate resursele și serviciile sistemului sunt expuse ca fișiere, permițând interacțiuni uniforme și izolarea proceselor.

Bibliografie

- [1] AMD, *AMD64 Architecture Programmer's Manual, Volume 3: General-Purpose and System Programming Instructions*, https://docs.amd.com/v/u/en-US/24594_3.37, 2025.
- [2] UEFI Forum, *Unified Extensible Firmware Interface (UEFI) Specification*, https://uefi.org/sites/default/files/resources/UEFI_Spec_Final_2.11.pdf, 2024.
- [3] Limine Project, *Limine Bootloader*, <https://codeberg.org/Limine/Limine>, 2025.
- [4] Rémy Card, Theodore Ts'o și Stephen Tweedie, *The Second Extended File System (EXT2)*, <https://www.kernel.org/doc/html/latest/filesystems/ext2.html>, 1994.
- [5] Tool Interface Standards Committee, *Executable and Linkable Format (ELF-64) Specification*, <https://refspecs.linuxfoundation.org/elf/elf.pdf>, 1995.
- [6] UNIX System Laboratories, *System V Application Binary Interface (AMD64 Architecture Processor Supplement)*, https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf, 2012.
- [7] Fabrice Bellard, *Tiny C Compiler (TCC)*, <https://bellard.org/tcc/>, 2025.